

NAME: SACHIN

ROLL NO.: 251256

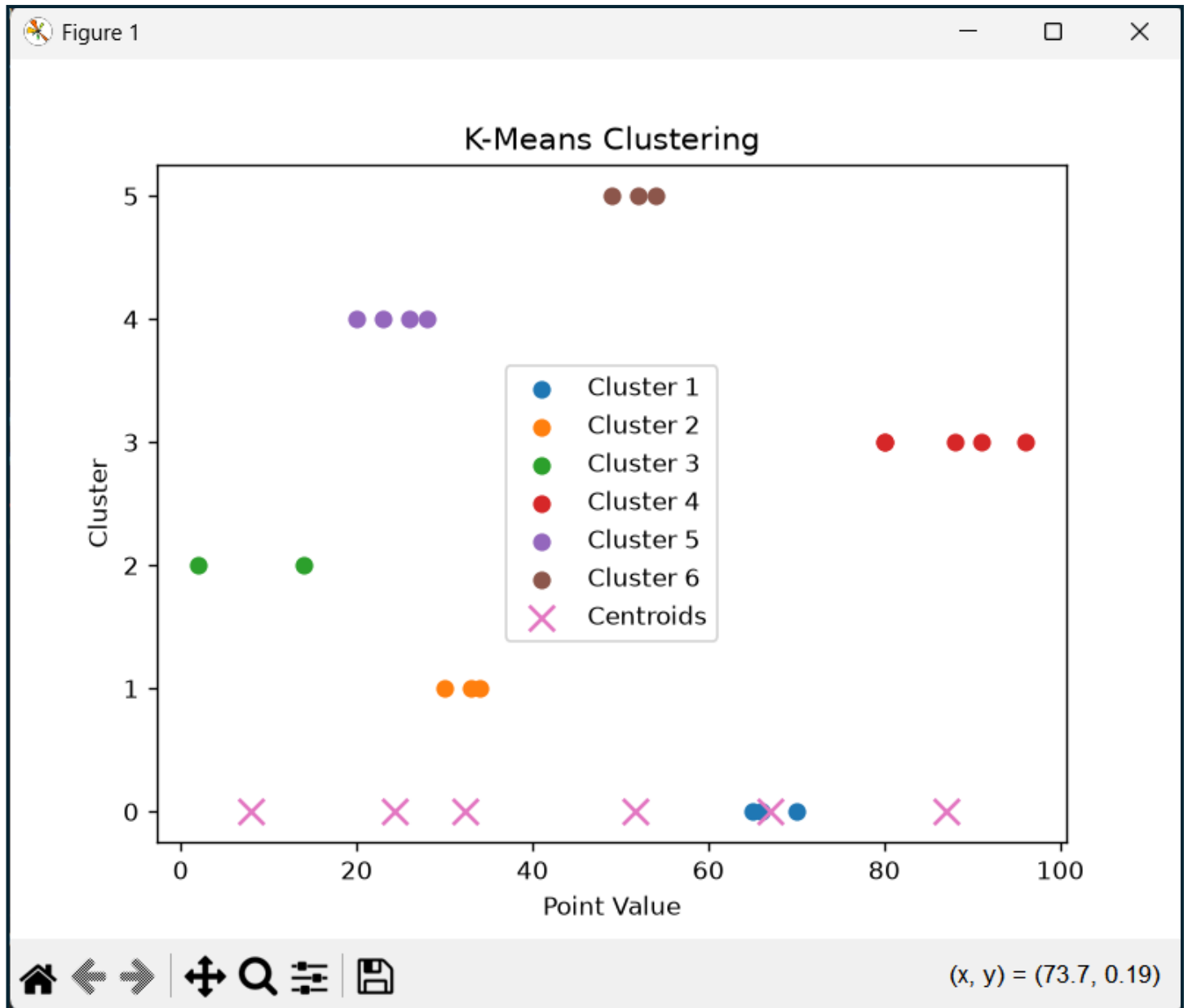
DEPT: BSC. (H.) COMPUTER
SCIENCE

SUBJECT: DSE- DATA
MINING (DM)

ASSIGNMENT

1. Apply K means over 20 random 1-dimension points with k = 3, ask the user to give values of k.

```
1  import random
2  import matplotlib.pyplot as plt
3
4  # Generate 20 random 1-D points
5  points = [random.randint(1, 100) for _ in range(20)]
6
7  print("Points:", points)
8
9  # Take k from user
10 k = int(input("Enter value of k: "))
11
12 # Randomly select initial centroids
13 centroids = random.sample(points, k)
14
15 for iteration in range(10):
16
17     # Create clusters
18     clusters = [[] for _ in range(k)]
19
20     for point in points:
21         distances = [abs(point - c) for c in centroids]
22         nearest = distances.index(min(distances))
23         clusters[nearest].append(point)
24
25     # Calculate new centroids
26     new_centroids = []
27
28     for cluster in clusters:
29         if cluster:
30             new_centroids.append(sum(cluster) / len(cluster))
31         else:
32             new_centroids.append(random.choice(points))
33
34     # Stop if centroids do not change
35     if new_centroids == centroids:
36         break
37
38     centroids = new_centroids
39
40 # Display result
41 print("\nFinal Centroids:", centroids)
42
43 for i in range(k):
44     print("Cluster", i + 1, ":", clusters[i])
45
46 # Plot clusters
47 for i in range(k):
48     plt.scatter(clusters[i], [i] * len(clusters[i]), label=f"Cluster {i+1}")
49
50 plt.scatter(centroids, [0] * k, marker="x", s=100, label="Centroids")
51
52 plt.xlabel("Point Value")
53 plt.ylabel("Cluster")
54 plt.title("K-Means Clustering")
55 plt.legend()
56 plt.show()
```



```

(.venv) PS D:\DM assignment 1> & "d:\DM assignment 1\.venv\Scripts\python.exe" "d:\DM assignment 1/Practical 4/Practical 5/Q1.py"
Points: [91, 34, 30, 65, 70, 14, 52, 23, 49, 28, 88, 96, 66, 20, 33, 54, 80, 80, 26, 2]
Enter value of k: 6

Final Centroids: [67.0, 32.333333333333336, 8.0, 87.0, 24.25, 51.666666666666664]
Cluster 1 : [65, 70, 66]
Cluster 2 : [34, 30, 33]
Cluster 3 : [14, 2]
Cluster 4 : [91, 88, 96, 80, 80]
Cluster 5 : [23, 28, 20, 26]
Cluster 6 : [52, 49, 54]
(.venv) PS D:\DM assignment 1>
  
```

2. Apply simple K-means algorithm for clustering any dataset. Compare the performance of clusters by varying the algorithm parameters. For a given set of parameters, plot a line graph depicting MSE obtained after each iteration.

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # Dataset
5  X = np.array([
6      [1, 2], [2, 1], [2, 3], [3, 2],
7      [8, 8], [9, 7], [8, 9], [7, 8],
8      [15, 2], [16, 3], [15, 4], [17, 2]
9  ])
10
11 # K-Means function
12 def kmeans(X, k, max_iterations=10):
13
14     # Select first k points as initial centroids
15     centroids = X[:k].astype(float)
16
17     mse_history = []
18
19     for iteration in range(max_iterations):
20
21         # Calculate distance of every point from every centroid
22         distances = np.zeros((len(X), k))
23
24         for i in range(len(X)):
25             for j in range(k):
26                 distances[i][j] = np.linalg.norm(X[i] - centroids[j])
27
28         # Assign points to nearest centroid
29         labels = np.argmin(distances, axis=1)
30
31         # Calculate MSE
32         mse = 0
33
34         for i in range(len(X)):
35             mse += np.sum((X[i] - centroids[labels[i]]) ** 2)
36
37         mse = mse / len(X)
38         mse_history.append(mse)
39
40         # Calculate new centroids
41         new_centroids = []
42
43         for j in range(k):
44             cluster = X[labels == j]
45
46             if len(cluster) > 0:
47                 new_centroids.append(np.mean(cluster, axis=0))
48             else:
49                 new_centroids.append(centroids[j])
50
51         new_centroids = np.array(new_centroids)
52
53         # Stop if centroids do not change
54         if np.allclose(centroids, new_centroids):
55             break
56
57         centroids = new_centroids
58
59     return labels, centroids, mse_history
60
61 # Take parameters from user
62 k = int(input("Enter number of clusters (k): "))
63 iterations = int(input("Enter maximum iterations: "))
```

```

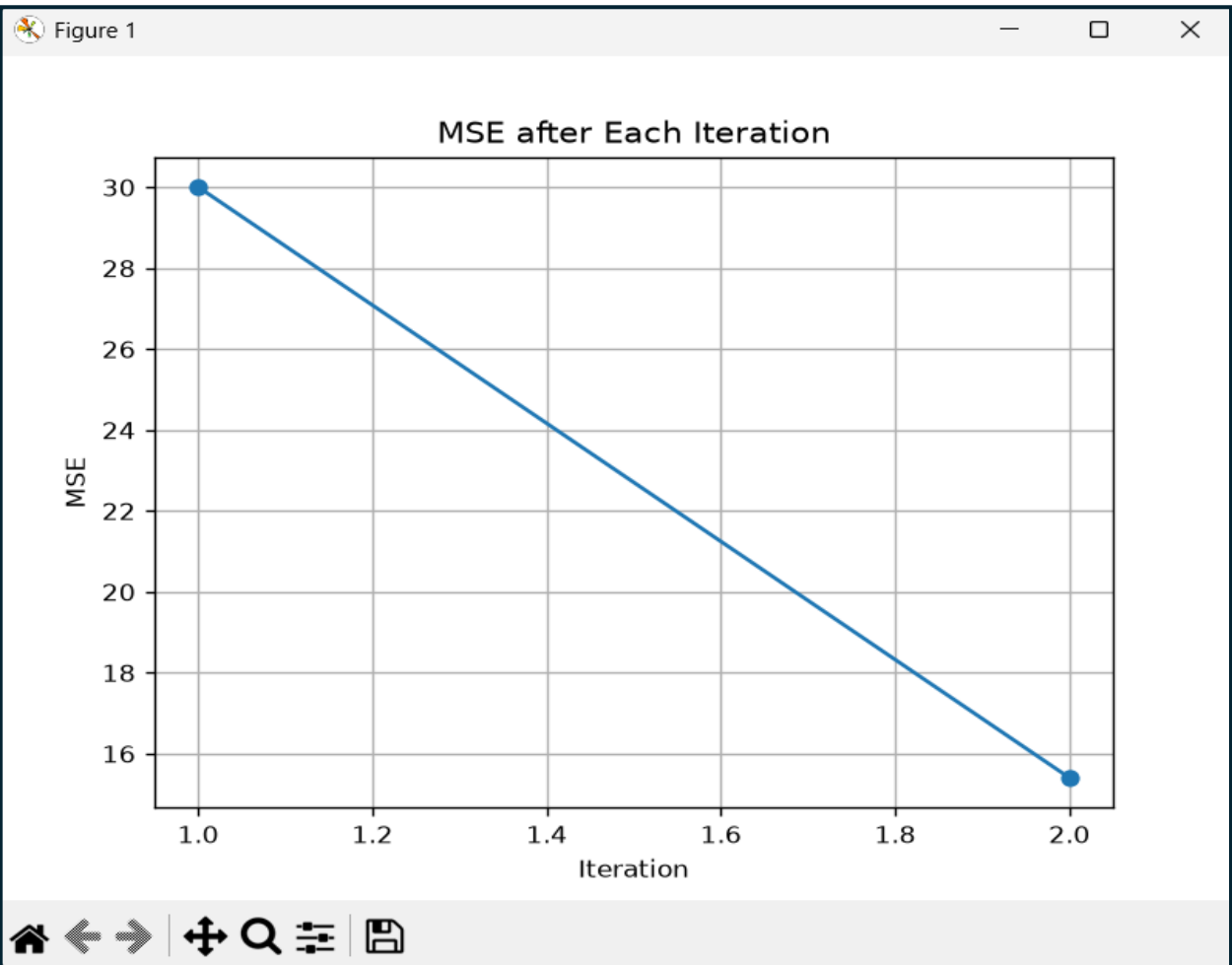
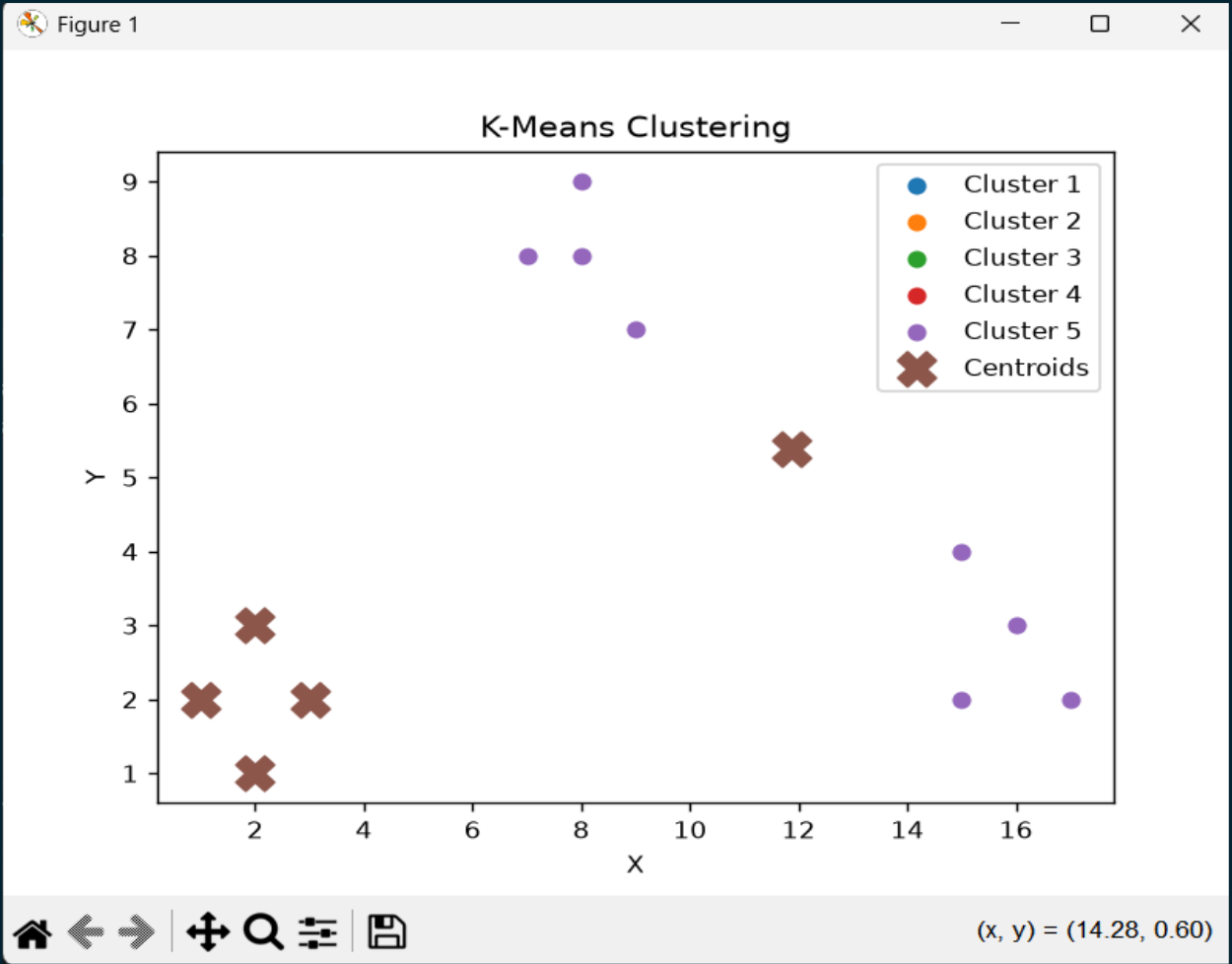
66 # Apply K-Means
67 labels, centroids, mse_history = kmeans(X, k, iterations)
68
69 print("\nFinal Centroids:")
70 print(centroids)
71
72 print("\nMSE after each iteration:")
73 for i, mse in enumerate(mse_history):
74     print("Iteration", i + 1, ":", mse)
75
76
77 # Plot clusters
78 plt.figure()
79
80 for i in range(k):
81     cluster = X[labels == i]
82     plt.scatter(cluster[:, 0], cluster[:, 1], label=f"Cluster {i+1}")
83
84 plt.scatter(
85     centroids[:, 0],
86     centroids[:, 1],
87     marker="X",
88     s=200,
89     label="Centroids"
90 )
91
92 plt.xlabel("X")
93 plt.ylabel("Y")
94 plt.title("K-Means Clustering")
95 plt.legend()
96 plt.show()
97

```

```

99 # Plot MSE vs iterations
100 plt.figure()
101
102 plt.plot(
103     range(1, len(mse_history) + 1),
104     mse_history,
105     marker="o"
106 )
107
108 plt.xlabel("Iteration")
109 plt.ylabel("MSE")
110 plt.title("MSE after Each Iteration")
111 plt.grid()
112
113 plt.show()

```



```
copy2 /
(.venv) PS D:\DM assignment 1> & "d:\DM assignment 1\.venv\Scripts\python.exe" "d:/DM assignment 1/Practical 4/Practical 5/Q2.py"
Enter number of clusters (k): 5
Enter maximum iterations: 7

Final Centroids:
[[ 1.  2. ]
 [ 2.  1. ]
 [ 2.  3. ]
 [ 3.  2. ]
 [11.875  5.375]]

MSE after each iteration:
Iteration 1 : 30.0
Iteration 2 : 15.395833333333334
(.venv) PS D:\DM assignment 1>
```

THANK YOU